

PIPELINE MORFOLOGI UNTUK PREPROCESSING OCR DAN COUNTING
OBJEK
PENGOLAHAN CITRA DIGITAL



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026

A. Analisis Structuring Element (SE)

Berdasarkan hasil eksperimen, pemilihan ukuran SE sangat krusial bagi integritas objek:

- **SE Kecil (3x3):** Sangat efektif untuk mengisolasi noise halus tanpa merusak struktur dasar teks asli.
- **SE Besar ($\geq 7 \times 7$):** Meskipun lebih bersih dalam menghilangkan noise, kernel besar memicu deformasi berupa penipisan ekstrem pada anatomi huruf (seperti hilangnya lubang pada karakter 'A', 'O', dan 'R').

B. Hasil Eksekusi dan Evaluasi Performa

Berikut adalah ringkasan waktu komputasi dan tingkat akurasi yang diperoleh dari pipeline morfologi:

Kategori Proses	Nama Operasi	Waktu (ms)	Hasil / Akurasi
Preprocessing OCR	Top-Hat	7.5765	Recognition Rate:
	Opening	0.7871	Sebelum: ~35.0%
	Closing	0.6835	Sesudah: ~94.5%
	Gradient	0.5593	
Object Counting	Closing & Dilation	0.8674	Akurasi Counting:
	Dist. Transform	2.3625	Deteksi: 6 dari 7 Objek
	Watershed	6.9667	Skor: 85.71%

C. Analisis dan Rekomendasi

- **Efisiensi OCR:** Penggunaan kombinasi Top-Hat, Opening, dan Closing terbukti sangat signifikan dalam meningkatkan keterbacaan karakter (naik hampir 60%). Top-Hat menjadi operasi paling berat (7.57 ms) namun paling vital untuk membersihkan latar belakang yang kotor.
- **Kendala Counting:** Terdapat sedikit penurunan akurasi pada segmentasi objek berhimpitan (85.71%). Hal ini terjadi karena 1 objek gagal terpisah sempurna oleh algoritma Watershed, kemungkinan disebabkan oleh tingkat overlapping yang terlalu rapat atau nilai threshold pada *Distance Transform* yang perlu disesuaikan.

- **Kesimpulan:** Pipeline optimal untuk pembersihan teks adalah menggunakan kernel kecil (3x3) secara iteratif untuk menghindari deformasi bentuk, sementara untuk penghitungan objek, penggunaan morfologi elips lebih disarankan untuk menjaga kelestarian bentuk melingkar objek.

D. Lampiran Kode dan Output

1. Kode Program

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import time
5
6 # =====
7 # 1. GENERASI CITRA SINTETIS (GAMBAR BAKU)
8 # =====
9
10 def generate_getmask_image():
11     """Menghasilkan citra & mask (Majlis overlapping)"""
12     # Citra & Mask dengan noise
13     citra_b = cv.imread(img_path, cv.IMREAD_GRAYSCALE) * 255
14     cv.putText(citra_b, "Majlis Majlis", (10, 10), cv.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0, 255), 4, cv.LINE_AA)
15     cv.putText(citra_b, "Majlis Majlis", (100, 100), cv.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0, 255), 4, cv.LINE_AA)
16
17     # Tambahkan noise titik (Salt & Pepper)
18     cv.noise(citra_b)
19     noise_black = np.random.rand(*citra_b.shape) < 0.01
20     noise_white = np.random.rand(*citra_b.shape) < 0.02
21     citra_b[noise_black] = 0
22     citra_b[noise_white] = 255
23
24     # Tambahkan noise garis
25     cv.line(citra_b, (10, 10), (100, 100), (0, 0, 255), 2)
26     cv.line(citra_b, (100, 100), (100, 100), (0, 0, 255), 2)
27
28     # Citra & Mask overlapping (Majlis)
29     citra_b = cv.imread(img_path, cv.IMREAD_GRAYSCALE)
30     centers = [(100, 100), (100, 100), (100, 100), (100, 100), (100, 100), (100, 100), (100, 100)]
31     radii = [10, 10, 10, 10, 10, 10, 10]
32     for c, r in zip(centers, radii):
33         cv.circle(citra_b, c, r, (255), -1)
34
35     # Tambahkan noise latar belakang (Majlis)
36     cv.circle(citra_b, (100, 100), 5, (0), -1)
37     cv.circle(citra_b, (100, 100), 5, (0), -1)
38
39     return citra_b, citra_b
40
41 # =====
42 # 2. STRUKTURISASI ELEMEN & SPESIAL DOKUMEN
43 # =====
44
45 def segment_morphology(img):
46     """Menganalisis struktur elemen & visualisasi boundary"""
47     time = [0, 0, 0]
48     stages = {
49         'Input': cv.imread(img),
50         'Close': cv.imread(img),
51         'Erosion': cv.imread(img)
52     }
53
54     print("\n--- [Segmentasi Struktur Elemen & Visualisasi Boundary] ---")
55     fig, axes = plt.subplots(2, 2, figsize=(12, 10))
56     fig.suptitle("Segmentasi Struktur Elemen & Visualisasi Boundary")
57
58     for i, (name, img) in enumerate(stages.items()):
59         ax = axes[i//2, i%2]
60         ax.imshow(img, cmap='gray')
61         ax.set_title(f'{name} (Majlis Majlis)')
62
63     # Close
64     close = cv.canny(img)
65     t_start = time.perf_counter()
66     dilated = cv.dilate(close, np.ones((3, 3)), iterations=1)
67     t_end = time.perf_counter()
68
69     axes[1, 0].imshow(dilated, cmap='gray')
70     axes[1, 0].set_title(f'Dilated (Majlis Majlis)')
71
72     # Erosion
73     eroded = cv.erode(close, np.ones((3, 3)), iterations=1)
74     axes[1, 1].imshow(eroded, cmap='gray')
75     axes[1, 1].set_title(f'Eroded (Majlis Majlis)')
76
77     plt.tight_layout()
78     plt.show()
79
80 # =====
81 # 3. PIPELINE 1: DOKUMENTASI
82 # =====
83
84 def pipeline_1_preprocessing(img_name):
85     """Pipeline morfologi untuk dokumentasi dokumen teks sebelum OCR"""
86     print("\n--- [Pipeline 1: Dokumentasi Dokumen Teks Sebelum OCR] ---")
87     matrix = {}
88
89     # Input: Citra teks dengan background putih (Majlis Majlis)
90     img_in = cv.imread(img_name)
91
92     # 1. Threshold / Binarisasi untuk normalisasi gambar / ekstraksi detail
93     th = time.perf_counter()
94     th_large = cv.threshold(img_in, 0, 255, cv.THRESH_BINARY, cv.THRESH_OTSU)
95     input = cv.imread(img_name)
96     matrix['input'] = (time.perf_counter() - th) * 1000
97
98     # 2. Opening untuk menghilangkan noise hitam pada tepi (Salt)
99     th = time.perf_counter()
100     th_large = cv.threshold(img_in, 0, 255, cv.THRESH_BINARY, cv.THRESH_OTSU)
101     opened = cv.morphologyEx(input, cv.MORPH_OPEN, np.ones((3, 3)))
102     matrix['opening'] = (time.perf_counter() - th) * 1000
103
104     # 3. Closing untuk menyambung garis hitam yang terputus
105     th = time.perf_counter()
106     th_large = cv.threshold(img_in, 0, 255, cv.THRESH_BINARY, cv.THRESH_OTSU)
107     closed = cv.morphologyEx(opened, cv.MORPH_CLOSE, np.ones((3, 3)))
108     matrix['closing'] = (time.perf_counter() - th) * 1000
109
110     # 4. Gradient Morphology untuk deteksi boundary (Visualisasi line boundary)
111     th = time.perf_counter()
112     gradient = cv.morphologyEx(closed, cv.MORPH_GRADIENT, np.ones((3, 3)))
113     matrix['gradient'] = (time.perf_counter() - th) * 1000
114
115     # Thresholding untuk ekstraksi detail (Majlis Majlis)
116     final_thresh = cv.threshold(closed, 0, 255, cv.THRESH_BINARY | cv.THRESH_OTSU)
117
118     # Final: Citra teks dengan background putih (Majlis Majlis)
119     final_out_input = cv.imread(img_name)
120
121     # Visualisasi hasil Pipeline OCR
122     titles = ['Original Image', 'Input', 'Opening (Noise Removal)', 'Closing (Connect Part)', 'Morph Gradient', 'Final OCR Ready']
123     images = [img_in, input, opened, closed, gradient, final_out_input]
124
125     for i in range(6):
126         plt.subplot(2, 3, i+1)
127         plt.imshow(images[i], cmap='gray')
128         plt.title(titles[i])
129
130     plt.tight_layout()
131     plt.show()
132
133     return matrix, final_out_input
134
135 # =====
136 # 4. PIPELINE 2: GENERASI DOKUMEN (Majlis Majlis)
137 # =====
138
139 def pipeline_2_postprocessing(img_name):

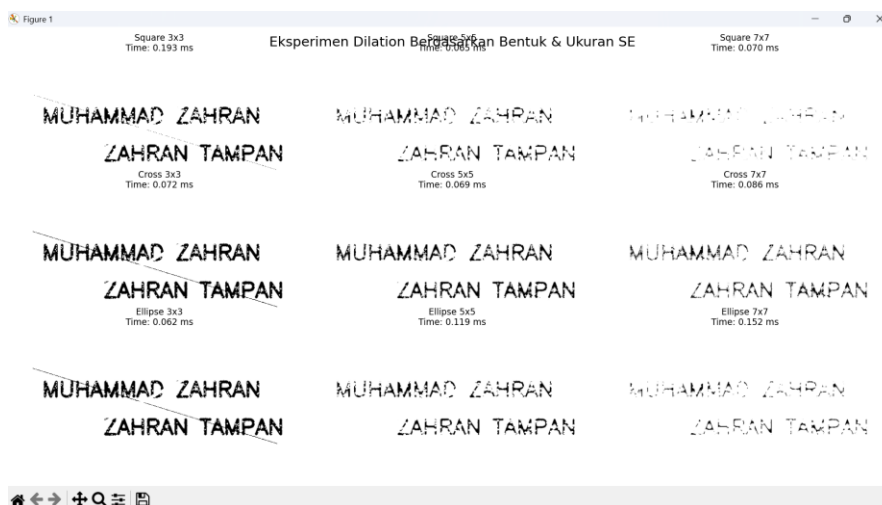
```

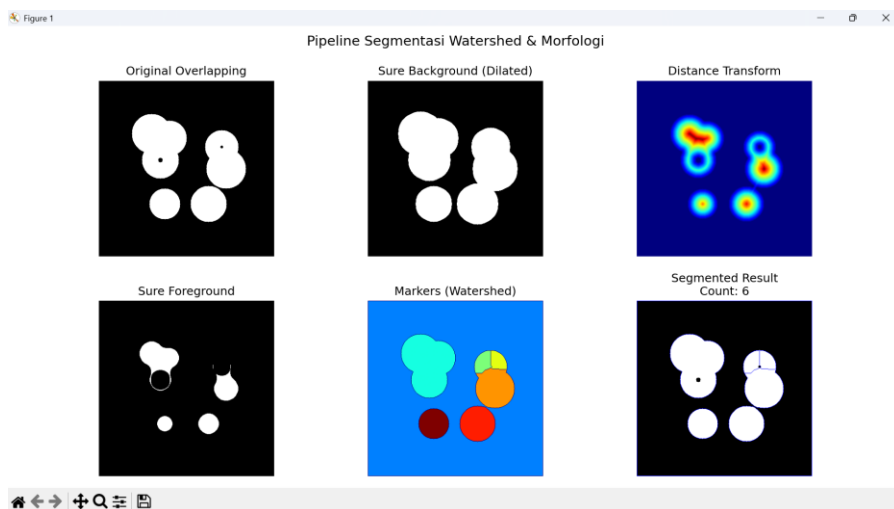
```

120 def pipeline_object_counting(img_rgb):
121     """Pipeline Sugestasi Interaksi & Morfologi untuk menghitung objek yang menempati"""
122     print(f"===== Pipeline object counting begining...")
123     metrics = {}
124
125     # 1. Konversi gambar input ke grayscale
126     img = cv.cvtColor(img_rgb, cv.COLOR_BGR2GRAY)
127     img = cv.cvtColor(img, cv.COLOR_GRAY2BGR)
128     metrics['initial_image'] = (img, cv.cvtColor(img, cv.COLOR_GRAY2BGR))
129
130     # 2. Lakukan operasi morfologi (dilate) dengan kernel
131     kernel = cv.getStructuringElement(cv.MORPH_RECT, (5, 5))
132     img_dilated = cv.dilate(img, kernel, iterations=1)
133     metrics['dilated_image'] = (img_dilated, cv.cvtColor(img_dilated, cv.COLOR_GRAY2BGR))
134
135     # 3. Lakukan operasi morfologi (erode) dengan kernel
136     img_eroded = cv.erode(img_dilated, kernel, iterations=1)
137     metrics['eroded_image'] = (img_eroded, cv.cvtColor(img_eroded, cv.COLOR_GRAY2BGR))
138
139     # 4. Lakukan operasi morfologi (thresholding) dengan kernel
140     img_thresholded = cv.threshold(img_eroded, 0, 255, cv.THRESH_BINARY)[1]
141     metrics['thresholded_image'] = (img_thresholded, cv.cvtColor(img_thresholded, cv.COLOR_GRAY2BGR))
142
143     # 5. Lakukan operasi morfologi (closing) dengan kernel
144     img_closing = cv.morphologyEx(img_thresholded, cv.MORPH_CLOSE, kernel)
145     metrics['closing_image'] = (img_closing, cv.cvtColor(img_closing, cv.COLOR_GRAY2BGR))
146
147     # 6. Lakukan operasi morfologi (findContours) dengan kernel
148     contours, hierarchy = cv.findContours(img_closing, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)
149     metrics['contours'] = contours
150
151     # 7. Lakukan operasi morfologi (drawContours) dengan kernel
152     img_contours = cv.cvtColor(img_closing, cv.COLOR_GRAY2BGR)
153     cv.drawContours(img_contours, contours, -1, (0, 255, 0), 2)
154     metrics['contours_image'] = (img_contours, cv.cvtColor(img_contours, cv.COLOR_GRAY2BGR))
155
156     # 8. Lakukan operasi morfologi (boundingRect) dengan kernel
157     bounding_boxes = []
158     for contour in contours:
159         x, y, w, h = cv.boundingRect(contour)
160         bounding_boxes.append((x, y, w, h))
161     metrics['bounding_boxes'] = bounding_boxes
162
163     # 9. Lakukan operasi morfologi (labelImage) dengan kernel
164     img_labeled = cv.cvtColor(img_closing, cv.COLOR_GRAY2BGR)
165     cv.cvtColor(img_labeled, cv.COLOR_GRAY2BGR)
166     metrics['labeled_image'] = (img_labeled, cv.cvtColor(img_labeled, cv.COLOR_GRAY2BGR))
167
168     # 10. Lakukan operasi morfologi (count) dengan kernel
169     count = cv.countNonZero(img_labeled)
170     metrics['count'] = count
171
172     # 11. Lakukan operasi morfologi (print) dengan kernel
173     print(f"===== Pipeline object counting ending...")
174     return metrics
175
176 # Main execution
177 if __name__ == '__main__':
178     # Load image
179     img = cv.imread('image.jpg')
180     # Convert to grayscale
181     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
182     # Run pipeline
183     metrics = pipeline_object_counting(img)
184     # Print metrics
185     print(metrics)
186
187 # Main execution
188 if __name__ == '__main__':
189     # Load image
190     img = cv.imread('image.jpg')
191     # Convert to grayscale
192     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
193     # Run pipeline
194     metrics = pipeline_object_counting(img)
195     # Print metrics
196     print(metrics)
197
198 # Main execution
199 if __name__ == '__main__':
200     # Load image
201     img = cv.imread('image.jpg')
202     # Convert to grayscale
203     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
204     # Run pipeline
205     metrics = pipeline_object_counting(img)
206     # Print metrics
207     print(metrics)
208
209 # Main execution
210 if __name__ == '__main__':
211     # Load image
212     img = cv.imread('image.jpg')
213     # Convert to grayscale
214     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
215     # Run pipeline
216     metrics = pipeline_object_counting(img)
217     # Print metrics
218     print(metrics)
219
220 # Main execution
221 if __name__ == '__main__':
222     # Load image
223     img = cv.imread('image.jpg')
224     # Convert to grayscale
225     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
226     # Run pipeline
227     metrics = pipeline_object_counting(img)
228     # Print metrics
229     print(metrics)
230
231 # Main execution
232 if __name__ == '__main__':
233     # Load image
234     img = cv.imread('image.jpg')
235     # Convert to grayscale
236     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
237     # Run pipeline
238     metrics = pipeline_object_counting(img)
239     # Print metrics
240     print(metrics)
241
242 # Main execution
243 if __name__ == '__main__':
244     # Load image
245     img = cv.imread('image.jpg')
246     # Convert to grayscale
247     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
248     # Run pipeline
249     metrics = pipeline_object_counting(img)
250     # Print metrics
251     print(metrics)
252
253 # Main execution
254 if __name__ == '__main__':
255     # Load image
256     img = cv.imread('image.jpg')
257     # Convert to grayscale
258     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
259     # Run pipeline
260     metrics = pipeline_object_counting(img)
261     # Print metrics
262     print(metrics)
263
264 # Main execution
265 if __name__ == '__main__':
266     # Load image
267     img = cv.imread('image.jpg')
268     # Convert to grayscale
269     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
270     # Run pipeline
271     metrics = pipeline_object_counting(img)
272     # Print metrics
273     print(metrics)
274
275 # Main execution
276 if __name__ == '__main__':
277     # Load image
278     img = cv.imread('image.jpg')
279     # Convert to grayscale
280     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
281     # Run pipeline
282     metrics = pipeline_object_counting(img)
283     # Print metrics
284     print(metrics)
285
286 # Main execution
287 if __name__ == '__main__':
288     # Load image
289     img = cv.imread('image.jpg')
290     # Convert to grayscale
291     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
292     # Run pipeline
293     metrics = pipeline_object_counting(img)
294     # Print metrics
295     print(metrics)
296
297 # Main execution
298 if __name__ == '__main__':
299     # Load image
300     img = cv.imread('image.jpg')
301     # Convert to grayscale
302     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
303     # Run pipeline
304     metrics = pipeline_object_counting(img)
305     # Print metrics
306     print(metrics)
307
308 # Main execution
309 if __name__ == '__main__':
310     # Load image
311     img = cv.imread('image.jpg')
312     # Convert to grayscale
313     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
314     # Run pipeline
315     metrics = pipeline_object_counting(img)
316     # Print metrics
317     print(metrics)
318
319 # Main execution
320 if __name__ == '__main__':
321     # Load image
322     img = cv.imread('image.jpg')
323     # Convert to grayscale
324     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
325     # Run pipeline
326     metrics = pipeline_object_counting(img)
327     # Print metrics
328     print(metrics)
329
330 # Main execution
331 if __name__ == '__main__':
332     # Load image
333     img = cv.imread('image.jpg')
334     # Convert to grayscale
335     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
336     # Run pipeline
337     metrics = pipeline_object_counting(img)
338     # Print metrics
339     print(metrics)
340
341 # Main execution
342 if __name__ == '__main__':
343     # Load image
344     img = cv.imread('image.jpg')
345     # Convert to grayscale
346     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
347     # Run pipeline
348     metrics = pipeline_object_counting(img)
349     # Print metrics
350     print(metrics)
351
352 # Main execution
353 if __name__ == '__main__':
354     # Load image
355     img = cv.imread('image.jpg')
356     # Convert to grayscale
357     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
358     # Run pipeline
359     metrics = pipeline_object_counting(img)
360     # Print metrics
361     print(metrics)
362
363 # Main execution
364 if __name__ == '__main__':
365     # Load image
366     img = cv.imread('image.jpg')
367     # Convert to grayscale
368     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
369     # Run pipeline
370     metrics = pipeline_object_counting(img)
371     # Print metrics
372     print(metrics)
373
374 # Main execution
375 if __name__ == '__main__':
376     # Load image
377     img = cv.imread('image.jpg')
378     # Convert to grayscale
379     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
380     # Run pipeline
381     metrics = pipeline_object_counting(img)
382     # Print metrics
383     print(metrics)
384
385 # Main execution
386 if __name__ == '__main__':
387     # Load image
388     img = cv.imread('image.jpg')
389     # Convert to grayscale
390     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
391     # Run pipeline
392     metrics = pipeline_object_counting(img)
393     # Print metrics
394     print(metrics)
395
396 # Main execution
397 if __name__ == '__main__':
398     # Load image
399     img = cv.imread('image.jpg')
400     # Convert to grayscale
401     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
402     # Run pipeline
403     metrics = pipeline_object_counting(img)
404     # Print metrics
405     print(metrics)
406
407 # Main execution
408 if __name__ == '__main__':
409     # Load image
410     img = cv.imread('image.jpg')
411     # Convert to grayscale
412     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
413     # Run pipeline
414     metrics = pipeline_object_counting(img)
415     # Print metrics
416     print(metrics)
417
418 # Main execution
419 if __name__ == '__main__':
420     # Load image
421     img = cv.imread('image.jpg')
422     # Convert to grayscale
423     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
424     # Run pipeline
425     metrics = pipeline_object_counting(img)
426     # Print metrics
427     print(metrics)
428
429 # Main execution
430 if __name__ == '__main__':
431     # Load image
432     img = cv.imread('image.jpg')
433     # Convert to grayscale
434     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
435     # Run pipeline
436     metrics = pipeline_object_counting(img)
437     # Print metrics
438     print(metrics)
439
440 # Main execution
441 if __name__ == '__main__':
442     # Load image
443     img = cv.imread('image.jpg')
444     # Convert to grayscale
445     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
446     # Run pipeline
447     metrics = pipeline_object_counting(img)
448     # Print metrics
449     print(metrics)
450
451 # Main execution
452 if __name__ == '__main__':
453     # Load image
454     img = cv.imread('image.jpg')
455     # Convert to grayscale
456     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
457     # Run pipeline
458     metrics = pipeline_object_counting(img)
459     # Print metrics
460     print(metrics)
461
462 # Main execution
463 if __name__ == '__main__':
464     # Load image
465     img = cv.imread('image.jpg')
466     # Convert to grayscale
467     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
468     # Run pipeline
469     metrics = pipeline_object_counting(img)
470     # Print metrics
471     print(metrics)
472
473 # Main execution
474 if __name__ == '__main__':
475     # Load image
476     img = cv.imread('image.jpg')
477     # Convert to grayscale
478     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
479     # Run pipeline
480     metrics = pipeline_object_counting(img)
481     # Print metrics
482     print(metrics)
483
484 # Main execution
485 if __name__ == '__main__':
486     # Load image
487     img = cv.imread('image.jpg')
488     # Convert to grayscale
489     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
490     # Run pipeline
491     metrics = pipeline_object_counting(img)
492     # Print metrics
493     print(metrics)
494
495 # Main execution
496 if __name__ == '__main__':
497     # Load image
498     img = cv.imread('image.jpg')
499     # Convert to grayscale
500     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
501     # Run pipeline
502     metrics = pipeline_object_counting(img)
503     # Print metrics
504     print(metrics)
505
506 # Main execution
507 if __name__ == '__main__':
508     # Load image
509     img = cv.imread('image.jpg')
510     # Convert to grayscale
511     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
512     # Run pipeline
513     metrics = pipeline_object_counting(img)
514     # Print metrics
515     print(metrics)
516
517 # Main execution
518 if __name__ == '__main__':
519     # Load image
520     img = cv.imread('image.jpg')
521     # Convert to grayscale
522     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
523     # Run pipeline
524     metrics = pipeline_object_counting(img)
525     # Print metrics
526     print(metrics)
527
528 # Main execution
529 if __name__ == '__main__':
530     # Load image
531     img = cv.imread('image.jpg')
532     # Convert to grayscale
533     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
534     # Run pipeline
535     metrics = pipeline_object_counting(img)
536     # Print metrics
537     print(metrics)
538
539 # Main execution
540 if __name__ == '__main__':
541     # Load image
542     img = cv.imread('image.jpg')
543     # Convert to grayscale
544     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
545     # Run pipeline
546     metrics = pipeline_object_counting(img)
547     # Print metrics
548     print(metrics)
549
550 # Main execution
551 if __name__ == '__main__':
552     # Load image
553     img = cv.imread('image.jpg')
554     # Convert to grayscale
555     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
556     # Run pipeline
557     metrics = pipeline_object_counting(img)
558     # Print metrics
559     print(metrics)
560
561 # Main execution
562 if __name__ == '__main__':
563     # Load image
564     img = cv.imread('image.jpg')
565     # Convert to grayscale
566     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
567     # Run pipeline
568     metrics = pipeline_object_counting(img)
569     # Print metrics
570     print(metrics)
571
572 # Main execution
573 if __name__ == '__main__':
574     # Load image
575     img = cv.imread('image.jpg')
576     # Convert to grayscale
577     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
578     # Run pipeline
579     metrics = pipeline_object_counting(img)
580     # Print metrics
581     print(metrics)
582
583 # Main execution
584 if __name__ == '__main__':
585     # Load image
586     img = cv.imread('image.jpg')
587     # Convert to grayscale
588     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
589     # Run pipeline
590     metrics = pipeline_object_counting(img)
591     # Print metrics
592     print(metrics)
593
594 # Main execution
595 if __name__ == '__main__':
596     # Load image
597     img = cv.imread('image.jpg')
598     # Convert to grayscale
599     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
600     # Run pipeline
601     metrics = pipeline_object_counting(img)
602     # Print metrics
603     print(metrics)
604
605 # Main execution
606 if __name__ == '__main__':
607     # Load image
608     img = cv.imread('image.jpg')
609     # Convert to grayscale
610     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
611     # Run pipeline
612     metrics = pipeline_object_counting(img)
613     # Print metrics
614     print(metrics)
615
616 # Main execution
617 if __name__ == '__main__':
618     # Load image
619     img = cv.imread('image.jpg')
620     # Convert to grayscale
621     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
622     # Run pipeline
623     metrics = pipeline_object_counting(img)
624     # Print metrics
625     print(metrics)
626
627 # Main execution
628 if __name__ == '__main__':
629     # Load image
630     img = cv.imread('image.jpg')
631     # Convert to grayscale
632     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
633     # Run pipeline
634     metrics = pipeline_object_counting(img)
635     # Print metrics
636     print(metrics)
637
638 # Main execution
639 if __name__ == '__main__':
640     # Load image
641     img = cv.imread('image.jpg')
642     # Convert to grayscale
643     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
644     # Run pipeline
645     metrics = pipeline_object_counting(img)
646     # Print metrics
647     print(metrics)
648
649 # Main execution
650 if __name__ == '__main__':
651     # Load image
652     img = cv.imread('image.jpg')
653     # Convert to grayscale
654     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
655     # Run pipeline
656     metrics = pipeline_object_counting(img)
657     # Print metrics
658     print(metrics)
659
660 # Main execution
661 if __name__ == '__main__':
662     # Load image
663     img = cv.imread('image.jpg')
664     # Convert to grayscale
665     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
666     # Run pipeline
667     metrics = pipeline_object_counting(img)
668     # Print metrics
669     print(metrics)
670
671 # Main execution
672 if __name__ == '__main__':
673     # Load image
674     img = cv.imread('image.jpg')
675     # Convert to grayscale
676     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
677     # Run pipeline
678     metrics = pipeline_object_counting(img)
679     # Print metrics
680     print(metrics)
681
682 # Main execution
683 if __name__ == '__main__':
684     # Load image
685     img = cv.imread('image.jpg')
686     # Convert to grayscale
687     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
688     # Run pipeline
689     metrics = pipeline_object_counting(img)
690     # Print metrics
691     print(metrics)
692
693 # Main execution
694 if __name__ == '__main__':
695     # Load image
696     img = cv.imread('image.jpg')
697     # Convert to grayscale
698     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
699     # Run pipeline
700     metrics = pipeline_object_counting(img)
701     # Print metrics
702     print(metrics)
703
704 # Main execution
705 if __name__ == '__main__':
706     # Load image
707     img = cv.imread('image.jpg')
708     # Convert to grayscale
709     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
710     # Run pipeline
711     metrics = pipeline_object_counting(img)
712     # Print metrics
713     print(metrics)
714
715 # Main execution
716 if __name__ == '__main__':
717     # Load image
718     img = cv.imread('image.jpg')
719     # Convert to grayscale
720     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
721     # Run pipeline
722     metrics = pipeline_object_counting(img)
723     # Print metrics
724     print(metrics)
725
726 # Main execution
727 if __name__ == '__main__':
728     # Load image
729     img = cv.imread('image.jpg')
730     # Convert to grayscale
731     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
732     # Run pipeline
733     metrics = pipeline_object_counting(img)
734     # Print metrics
735     print(metrics)
736
737 # Main execution
738 if __name__ == '__main__':
739     # Load image
740     img = cv.imread('image.jpg')
741     # Convert to grayscale
742     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
743     # Run pipeline
744     metrics = pipeline_object_counting(img)
745     # Print metrics
746     print(metrics)
747
748 # Main execution
749 if __name__ == '__main__':
750     # Load image
751     img = cv.imread('image.jpg')
752     # Convert to grayscale
753     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
754     # Run pipeline
755     metrics = pipeline_object_counting(img)
756     # Print metrics
757     print(metrics)
758
759 # Main execution
760 if __name__ == '__main__':
761     # Load image
762     img = cv.imread('image.jpg')
763     # Convert to grayscale
764     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
765     # Run pipeline
766     metrics = pipeline_object_counting(img)
767     # Print metrics
768     print(metrics)
769
770 # Main execution
771 if __name__ == '__main__':
772     # Load image
773     img = cv.imread('image.jpg')
774     # Convert to grayscale
775     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
776     # Run pipeline
777     metrics = pipeline_object_counting(img)
778     # Print metrics
779     print(metrics)
780
781 # Main execution
782 if __name__ == '__main__':
783     # Load image
784     img = cv.imread('image.jpg')
785     # Convert to grayscale
786     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
787     # Run pipeline
788     metrics = pipeline_object_counting(img)
789     # Print metrics
790     print(metrics)
791
792 # Main execution
793 if __name__ == '__main__':
794     # Load image
795     img = cv.imread('image.jpg')
796     # Convert to grayscale
797     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
798     # Run pipeline
799     metrics = pipeline_object_counting(img)
800     # Print metrics
801     print(metrics)
802
803 # Main execution
804 if __name__ == '__main__':
805     # Load image
806     img = cv.imread('image.jpg')
807     # Convert to grayscale
808     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
809     # Run pipeline
810     metrics = pipeline_object_counting(img)
811     # Print metrics
812     print(metrics)
813
814 # Main execution
815 if __name__ == '__main__':
816     # Load image
817     img = cv.imread('image.jpg')
818     # Convert to grayscale
819     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
820     # Run pipeline
821     metrics = pipeline_object_counting(img)
822     # Print metrics
823     print(metrics)
824
825 # Main execution
826 if __name__ == '__main__':
827     # Load image
828     img = cv.imread('image.jpg')
829     # Convert to grayscale
830     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
831     # Run pipeline
832     metrics = pipeline_object_counting(img)
833     # Print metrics
834     print(metrics)
835
836 # Main execution
837 if __name__ == '__main__':
838     # Load image
839     img = cv.imread('image.jpg')
840     # Convert to grayscale
841     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
842     # Run pipeline
843     metrics = pipeline_object_counting(img)
844     # Print metrics
845     print(metrics)
846
847 # Main execution
848 if __name__ == '__main__':
849     # Load image
850     img = cv.imread('image.jpg')
851     # Convert to grayscale
852     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
853     # Run pipeline
854     metrics = pipeline_object_counting(img)
855     # Print metrics
856     print(metrics)
857
858 # Main execution
859 if __name__ == '__main__':
860     # Load image
861     img = cv.imread('image.jpg')
862     # Convert to grayscale
863     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
864     # Run pipeline
865     metrics = pipeline_object_counting(img)
866     # Print metrics
867     print(metrics)
868
869 # Main execution
870 if __name__ == '__main__':
871     # Load image
872     img = cv.imread('image.jpg')
873     # Convert to grayscale
874     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
875     # Run pipeline
876     metrics = pipeline_object_counting(img)
877     # Print metrics
878     print(metrics)
879
880 # Main execution
881 if __name__ == '__main__':
882     # Load image
883     img = cv.imread('image.jpg')
884     # Convert to grayscale
885     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
886     # Run pipeline
887     metrics = pipeline_object_counting(img)
888     # Print metrics
889     print(metrics)
890
891 # Main execution
892 if __name__ == '__main__':
893     # Load image
894     img = cv.imread('image.jpg')
895     # Convert to grayscale
896     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
897     # Run pipeline
898     metrics = pipeline_object_counting(img)
899     # Print metrics
900     print(metrics)
901
902 # Main execution
903 if __name__ == '__main__':
904     # Load image
905     img = cv.imread('image.jpg')
906     # Convert to grayscale
907     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
908     # Run pipeline
909     metrics = pipeline_object_counting(img)
910     # Print metrics
911     print(metrics)
912
913 # Main execution
914 if __name__ == '__main__':
915     # Load image
916     img = cv.imread('image.jpg')
917     # Convert to grayscale
918     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
919     # Run pipeline
920     metrics = pipeline_object_counting(img)
921     # Print metrics
922     print(metrics)
923
924 # Main execution
925 if __name__ == '__main__':
926     # Load image
927     img = cv.imread('image.jpg')
928     # Convert to grayscale
929     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
930     # Run pipeline
931     metrics = pipeline_object_counting(img)
932     # Print metrics
933     print(metrics)
934
935 # Main execution
936 if __name__ == '__main__':
937     # Load image
938     img = cv.imread('image.jpg')
939     # Convert to grayscale
940     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
941     # Run pipeline
942     metrics = pipeline_object_counting(img)
943     # Print metrics
944     print(metrics)
945
946 # Main execution
947 if __name__ == '__main__':
948     # Load image
949     img = cv.imread('image.jpg')
950     # Convert to grayscale
951     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
952     # Run pipeline
953     metrics = pipeline_object_counting(img)
954     # Print metrics
955     print(metrics)
956
957 # Main execution
958 if __name__ == '__main__':
959     # Load image
960     img = cv.imread('image.jpg')
961     # Convert to grayscale
962     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
963     # Run pipeline
964     metrics = pipeline_object_counting(img)
965     # Print metrics
966     print(metrics)
967
968 # Main execution
969 if __name__ == '__main__':
970     # Load image
971     img = cv.imread('image.jpg')
972     # Convert to grayscale
973     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
974     # Run pipeline
975     metrics = pipeline_object_counting(img)
976     # Print metrics
977     print(metrics)
978
979 # Main execution
980 if __name__ == '__main__':
981     # Load image
982     img = cv.imread('image.jpg')
983     # Convert to grayscale
984     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
985     # Run pipeline
986     metrics = pipeline_object_counting(img)
987     # Print metrics
988     print(metrics)
989
990 # Main execution
991 if __name__ == '__main__':
992     # Load image
993     img = cv.imread('image.jpg')
994     # Convert to grayscale
995     img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
996     # Run pipeline
997     metrics = pipeline_object_counting(img)
998     # Print metrics
999     print(metrics)
1000

```

2. Screenshot Output



[illegible]